ARK-463 Preregistration — Production Deployment · Exact-Action Binding

Experiment ID: ARK-463
Series: P01 — Production Boundary Integrations
Action Type: Production Deployment
Failure Mode: Exact-Action Binding
Preregistration Date: 2026-07-18
Status: LOCKED (awaiting execution)

Research Question

Does an ExecutionProof-based production deployment guard correctly enforce exact-action binding across all deployment tuple dimensions, preventing authorization substitution or fuzzy matching?

A production deployment authorization must bind to an exact 5-tuple:
- `service_name` (string, case-sensitive)
- `environment` (string: production, staging, development, etc.)
- `version` (string: semantic version or image SHA)
- `region` (string: cloud region identifier)
- `deployment_method` (string: rolling-update, blue-green, canary, recreate)

Hypothesis: A correctly implemented deployment guard will:
1. ALLOW when the presented deployment tuple matches the authorized tuple exactly on all 5 dimensions
2. DENY when ANY dimension differs, even by a single character (case, whitespace, punctuation, prefix/suffix)
3. Never apply normalization, fuzzy matching, prefix matching, or wildcard expansion

---

Experimental Design

Test Structure

- **8 arms × 100 scenarios per arm = 800 total decisions**
- **Dual independent guards:** V1 (JavaScript) + V2 (Python)
- **Kill-gate control:** Broken guard that ignores exact-match requirement (always ALLOWs)

Arms (Decision Paths)

Arm 1: Exact Match (ALLOW)
All 5 dimensions match exactly. Expected: ALLOW (100/100)

Arm 2: Service Name Mismatch (DENY)
Service name differs (case, prefix, typo). Expected: DENY (100/100)

Arm 3: Environment Mismatch (DENY)
Environment differs (production vs staging, case variants). Expected: DENY (100/100)

Arm 4: Version Mismatch (DENY)
Version differs (v2.3.1 vs v2.3.2, SHA variants). Expected: DENY (100/100)

Arm 5: Region Mismatch (DENY)
Region differs (us-east-1 vs us-west-1, case). Expected: DENY (100/100)

Arm 6: Deployment Method Mismatch (DENY)
Deployment method differs (rolling-update vs blue-green). Expected: DENY (100/100)

Arm 7: Multiple Dimension Mismatch (DENY)
2+ dimensions differ simultaneously. Expected: DENY (100/100)

Arm 8: Exact Match Stress (ALLOW)
Exact matches with edge-case values (long names, special chars, SHAs). Expected: ALLOW (100/100)

Scenario Data Model

```
{
  "scenario_id": "string (arm_X_NNN format)",
  "authorized_deployment": {
    "service_name": "string",
    "environment": "string",
    "version": "string",
    "region": "string",
    "deployment_method": "string"
  },
  "presented_deployment": {
    "service_name": "string",
    "environment": "string",
    "version": "string",
    "region": "string",
    "deployment_method": "string"
  },
  "expected_decision": "ALLOW | DENY",
  "arm": "integer (1-8)",
  "mismatch_dimensions": ["array of dimension names"] // empty for exact match
}
```

Guard Implementations

V1 Guard (JavaScript - verifiers/v1_guard.js)

- Exact byte-equality comparison on all 5 dimensions
- Returns `{decision: "ALLOW" | "DENY", reason: string}`
- No normalization, no fuzzy matching

V2 Guard (Python - verifiers/v2_guard.py)

- Independent implementation in Python

- Exact string equality (`==`) on all 5 dimensions
- Returns `{decision: "ALLOW" | "DENY", reason: string}`

Kill-Gate (Broken Guard)

- Ignores exact-match requirement
- Always returns ALLOW regardless of tuple match
- Expected to produce wrong-allows on all DENY scenarios

Success Criteria (PASS Thresholds)

Primary Metrics:
- **C1: g_allow_exact ≥ 0.95** — Fraction of exact-match scenarios that ALLOW (Arms 1, 8)
- **C2: g_deny_mismatch_min ≥ 0.95** — Minimum fraction of mismatch scenarios that DENY (Arms 2-7, per-arm min)
- **C3: g_margin ≥ 0.90** — Safety margin: `min(g_allow_exact, g_deny_mismatch_min)`

Dual-Guard Concordance:
- **C4: Concordance ≥ 95%** — V1 and V2 must agree on ≥95% of all 800 scenarios

Kill-Gate Falsifiability:
- **C5: Kill-gate produces ≥ 50 wrong-allows** — Demonstrates testbed can detect broken guards

Verdict:
- **PASS** if C1 AND C2 AND C3 AND C4 AND C5 all met
- **FAIL** otherwise

Execution Protocol

1. **Lock:** Compute SHA-256 hashes of all source files → `MANIFEST.txt`
2. **Generate:** Run `generator/scenario_generator.py` to create 800 scenarios
3. **Execute Arms:** Run `run_arms.py` to test both guards on all 800 scenarios
4. **Execute Kill-Gate:** Run `run_killgate.py` to verify testbed falsifiability
5. **Record:** Save all results to `results/` directory (JSON format)
6. **Report:** Generate `RESULTS.md` with verdict and metrics
7. **Publish:** Commit all artifacts, create PR, verify, then publish to Zenodo

---

Compliance & Scope

RF Standing Covenant Compliance:
- ✓ Preregistration before execution
- ✓ Cryptographic lock (MANIFEST.txt)
- ✓ All outcomes preserved (PASS/FAIL)
- ✓ No legal/patent claims
- ✓ Synthetic data only
- ✓ Results published regardless of outcome

Limitations:
- This is a testbed experiment, NOT production-ready authorization
- Does not test network security, credential management, or audit logging
- Does not validate deployment safety, rollback logic, or health checks
- Exact-action binding is necessary but NOT sufficient for production security

---

Predicted Outcome

Hypothesis: Both guards will achieve g_allow_exact = 1.0000 and g_deny_mismatch_min = 1.0000, resulting in VERDICT: PASS with 100% dual-guard concordance and complete kill-gate falsifiability.

Rationale: Exact string equality is a solved problem in both JavaScript and Python. The testbed design explicitly avoids edge cases that would require normalization or locale-aware comparison.

Preregistration locked: 2026-07-18
Execution: Pending MANIFEST.txt lock